

Site Safety Compliance Detector — technical memo

Kanagavel A K · RAP pre-hackathon screening · github.com/KanagavelAK/ppe-rt detr

1. Domain and dataset

Problem. Detect construction workers and whether each wears a hard hat, so compliance is monitored continuously from cameras already on site. The question a safety officer asks is not "how many helmets" but "is anyone without one".

Classes. helmet (a head with a helmet on), head (a bare head), person. Helmet and head are not COCO classes.

Source. Safety Helmet Detection, [andrewmvd/hard-hat-detection](https://kaggle.com/andrewmvd/hard-hat-detection) (Kaggle), 5,000 images, Pascal VOC XML. Licence: <copy from the dataset page>. No relabelling; `scripts/prepare_data.py` converts to YOLO and folds the aliases hat / hard-hat into helmet. **Evaluation only:** SH17 ([mugheesahmad/sh17-dataset-for-ppe-detection](https://kaggle.com/mugheesahmad/sh17-dataset-for-ppe-detection), CC BY-NC-SA 4.0), stock photography, 17 classes remapped to these three. Never trained on.

2. Split strategy

Image-level, `md5(file_stem)` bucketed 15 / 15 / 70 into test / val / train. The dataset has near-duplicate frames, so a random split would put near-identical images on both sides and inflate test mAP; and because the assignment is a pure function of the filename, re-running or adding data can never move an image from train into test. The repo's samples/ were chosen by the same rule.

Split	Images	helmet	head	person
train	3,447	12,908	4,089	497
val	798	3,086	811	186
test	755	2,972	885	68

Balance is **26 : 8 : 1**. head, the class that matters most (a missed head is a missed violation), is well represented. person is not: the source draws a person box on ~3 percent of workers, the most consequential fact about this data.

3. Metrics, and what they do not tell you

	mAP@50	mAP@50-95	P	R
In-domain test, 755 images	0.627	0.412	0.628	0.617
SH17 out-of-distribution, 600 images	0.026	0.011	0.053	0.036

In-domain per class	P	R	mAP@50
helmet	0.927	0.930	0.959
head	0.877	0.890	0.914
person	0.078	0.029	0.008

What the headline hides. 0.627 averages two strong classes and one that failed. The non-COCO classes sit at 0.96 / 0.91. person collapsed, not from capacity: with a person box on ~3 percent of workers, a correct person prediction is penalised 97 percent of the time, so the model stopped making them. The confusion matrix sends person ground truth to background, not to another class: a labelling-convention problem.

What the OOD number does not tell you. SH17 is large, centred stock subjects versus small heads in 416 px site shots, so the 0.60 gap is real domain shift. But it is inflated by a *label-definition* mismatch: SH17 labels head on helmeted heads too, whereas here head means bare only, so a correct model scores zero on that class by construction. person scores *higher* OOD (0.077) because SH17 labels every person. The OOD figure is a lower bound on generalisation, not a measurement of it.

What neither measures. mAP is a detection metric; the product question is a compliance relation between boxes. The operating metric is recall on head (0.89: one bare head in nine is missed). Compliance-level accuracy was not measured: the data has no per-worker compliance labels, and deriving them from the boxes would score the association rule against itself.

4. Five failure cases

Mined by `scripts/failure_cases.py` on the test split (conf 0.25, IoU 0.5), which measures every miss for scale, blur, overlap and luminance; 412 of 755 test images have at least one error. Annotated images: `artifacts/failures/`. Each hypothesis below was confirmed by eye.

1. workers360.png, night crowd, 19 errors. Four head misses, each 13-17 px (0.10-0.14 % of image area), three with mean luminance 34-58 (dark); sharpness is fine (Laplacian var 700-2500). Root cause: scale and exposure together, at the limit of a 640 px input. Also two pairs of near-identical head boxes (0.665/0.267 and 0.414/0.326 at the same pixels): RT-DETR has no NMS, so a second query fires on the same object at lower confidence. Fix: higher input resolution or tiling for wide night shots; a 0.45 threshold removes every duplicate here.

2. workers3822.png, lunch break, 15 errors. A white rice bowl in a basket is called helmet at 0.663 (box [165, 357, 194, 386]). Root cause: colour and shape confusion; the model has learned "round, white, top-lit" more than "on a head". Nine of the other FPs sit in the top 13 px or left 10 px of the frame, which is a mirrored padding strip (see pattern below).

3. workers210.png, safety briefing, 14 errors, zero misses. Eight person boxes at 0.27-0.43 on eight real workers with no person ground truth, and five helmet boxes in the top 14 px mirrored strip. Root cause: the labelling convention. Everything the model found is really there; the metric calls it wrong because the annotators did not draw it.

4. workers2095.png, floodlit rebar deck, 13 errors. Three helmet misses are all ground-truth boxes 5-11 px tall at $y = 0$, inside the mirrored strip: here the strip *was* annotated, in cases 2 and 3 it was not. Root cause: the dataset labels its own padding inconsistently, so no threshold is right for both. Two more duplicate pairs (0.594/0.432, 0.558/0.418). Fix: crop the reflected border in `prepare_data.py` before training.

5. workers506.png, rescue scene, 13 errors, 3 ground-truth boxes. The one miss is a helmet cut by the bottom frame edge (60 x 24 px, box touches $y = 414/416$); the model returned it as two fragments at 0.48 and 0.43, neither reaching IoU 0.5. Root cause: truncation. The nine head "false positives" at up to 0.84 are backs of heads on unlabelled crouching rescuers, correct under this dataset's definition of head.

Pattern. Genuine model errors are few and specific: objects under 0.15 % of the image, objects cut by the frame, one round-white-object confusion, and two helmet-to-head confusions at small scale (workers429, workers2737). Most of the error *count* is the ground truth: unlabelled people in crowds, no person boxes, and an inconsistently labelled mirrored border. Two things follow. Self-reported precision understates the model. And the API's 0.45 guardrail floor is evidence-based: every duplicate box in these eight images scored below it.

5. The reasoning layer

POST /ask runs four hand-written functions in order; no framework anywhere.

Route. Deterministic rules over a closed three-class vocabulary: faster than a model call and unable to hallucinate a route. Two kinds skip the detector: not about the image (*capital of France*) and about the image but outside the class list (*what colour is the truck*). The second matters more; a system that only checked "is this about the image" would run the detector and invent an answer from boxes that cannot support one.

Structure. `build_scene()` turns boxes into workers. A helmet box is a head with a helmet on and a head box is a bare head, so each is one worker's status. Person boxes, when present, refine that: a helmet ≥ 55 percent inside a person box but with its centre below the top 45 percent is being carried and is not credited; a person with no headgear in their head zone is unknown, never compliant and never a violation.

Guard. Deterministic, before any model: refuse on no detections, on all detections below 0.45, and for compliance questions on any unknown worker. The model is never asked to rate its own confidence.

Worked insufficient-information case (`tests/test_reasoning.py`). Three people detected; two have a helmet in their head zone; the third is a 0.71 person with nothing resolvable on their head. Answer: *"I do not have enough information to answer that confidently. 1 of 3 detected people have no helmet and no bare head associated with them, most likely because their head is occluded, cropped or too small to resolve."* Both alternatives would be wrong: "two of three compliant" hides a worker; "one in violation" invents one.

Compose. Only after the guard passes, one direct Messages API call phrases the facts; with no key, templates phrase the same facts and `answer_source` says so.

6. Reproducibility (`artifacts/training_receipt.json`)

Kaggle free tier, 2 x Tesla T4 15 GB, DDP; Python 3.12.13, torch 2.10.0+cu128, Ultralytics 8.3.40. `rtddetr-1.pt` COCO-pretrained, 40 epochs, batch 16 (8 per GPU), `imgsz 640`, AdamW, `lr 1e-4`, AMP, seed 0, patience 12. **Wall-clock 6,168 s (1.71 h).** Weights `best.pt` 66 MB: github.com/KanagavelAK/ppe-rtddetr/releases/download/v1.0/best.pt (or `python scripts/download_weights.py`). Exact steps: `notebooks/kaggle_ppe_rtddetr.ipynb`, Save & Run All, no datasets attached. **Live API and demo:** kanagavel-ppe-rtddetr.hf.space (Swagger at `/api/docs`).

7. What I tried first and changed

- P100 → T4 x2.** Kaggle's torch 2.10 dropped `sm_60` kernels; the P100 failed at `model.to(device)`. The notebook now probes compute capability.
- Ray Tune crash.** Ultralytics auto-registers a Ray callback whose private API Kaggle's newer ray removed; training died after epoch 1 until disabled.
- Single GPU → DDP + RAM cache:** 1.71 h instead of an estimated 3+.
- Compliance anchored on person boxes → per headgear box.** The first design needed a person box per worker; with person recall at 0.03 it refused nearly every real question ("no people detected" with four helmets in view). The rule was inverted: headgear defines the worker, person boxes only refine, and the refusal stays where it is honest (a visible person with no resolvable head). The most useful thing the evaluation taught me.